LangGraph: state machines para agentes

Máster AI Engineer · Sesión 5 · Ignacio de Pastors

thePower Tech School

AGENDA

01 Bloque teórico
02 Bloque práctico
03 Ejercicio para casa
04 Cierre y dudas



thePower
Tech School

01

Bloque teórico · 30 min

thePower
Tech School

Por qué LangGraph: cuando las cadenas se quedan cortas

Una cadena LCEL es lineal: A va a B va a C. Funciona hasta que el flujo tiene que decidir o repetir.

Sin bucles
Si el JSON sale mal, no lo reintenta.

Sin decisiones
No elige rama según lo que responde.

LangGraph: grafo de estados
Nodos, aristas y ciclos: el flujo decide.

CASO REAL Agente de soporte: consulta la BBDD y, si no halla el pedido, reformula y reintenta.

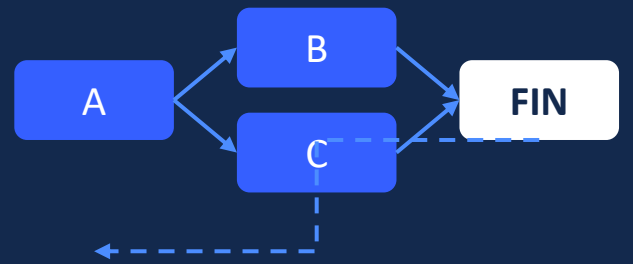
ANALOGÍA

Una cadena es una vía de tren recta. Un grafo es un mapa de metro: cruces, transbordos y vueltas.

CADENA · VÍA DE TREN



GRAFO · MAPA DE METRO



la flecha que vuelve = el bucle del agente

thePower Tech School

Los tres bloques de LangGraph



STATE
la mochila de datos que viaja por todo el grafo

EJEMPLO mensajes, numero_pedido, intentos



NODES
funciones que leen el state y devuelven su actualización

EJEMPLO buscar_en_bbdd(state)



EDGES
las flechas: qué nodo se ejecuta después de cuál

EJEMPLO START va a modelo va a END

State + Nodes + Edges: con estas tres piezas se construye cualquier agente, por complejo que sea.

CASO REAL Chatbot de pedidos: el State guarda la conversación, un Node consulta la BBDD y una Edge condicional decide si hay que volver al modelo.

thePower Tech School

El State: la memoria compartida del grafo

Un TypedDict que viaja por todo el grafo. Todo lo que el agente recuerda vive ahí dentro.

Cada nodo lo lee y lo actualiza
Devuelve solo las claves que cambia.

add_messages acumula
Suma mensajes en vez de machacarlos.

Única fuente de verdad
Si no está en el State, el grafo no lo sabe.

CASO REAL El agente de pedidos lleva mensajes, numero_pedido e intentos en un solo objeto.

CÓDIGO

```
class State(TypedDict):
    mensajes: Annotated[list, add_messages]
    numero_pedido: str
```

✖ Sin add_messages: cada nodo borra la lista.

✚ Con add_messages: la conversación se acumula.

IDEA CLAVE El State es la mochila: cada nodo la abre, mete lo suyo y la pasa al siguiente.

thePower Tech School

StateGraph paso a paso



1. Definir
el State: qué datos recuerda

StateGraph(State)



2. Añadir
nodos: una función por tarea

add_node()



3. Conectar
edges + START y END

add_edge()



4. Compilar
y ejecutar con invoke

compile()

EN CÓDIGO g = StateGraph(State) · g.add_node("modelo", llamar_al_modelo) · g.add_edge(START, "modelo")
g.add_edge("modelo", END) · app = g.compile() · app.invoke({"mensajes": [pregunta]})

El grafo compilado se invoca igual que una cadena: con invoke, y con stream si quieres ver el paso a paso.

thePower Tech School

Aristas condicionales: el grafo decide el camino

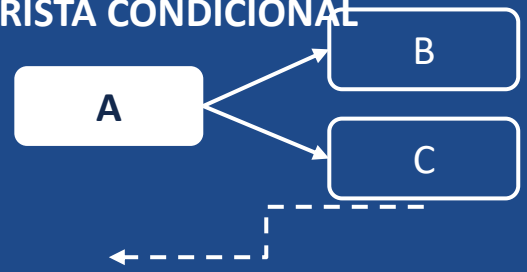
ARISTA NORMAL



- Siempre va de A a B: destino fijo.
- Flujo predecible y fácil de leer.
- Es lo mismo que una cadena LCEL.

EN CÓDIGO g.add_edge("tools", "modelo")

ARISTA CONDICIONAL



decide el router

- Una función mira el state y decide.
- Es un if/else dentro del flujo.
- **Permite bucles: sin esto no hay agente.**

EN CÓDIGO g.add_conditional_edges("modelo", decidir)

thePower Tech School

Persistencia: checkpointer y memoria

El checkpointer guarda el state después de cada paso.

thread_id = una conversación. Mismo id, sigue donde lo dejaste; id nuevo, chat en blanco.

Memoria y reanudar. Si el proceso se cae, retoma desde el último checkpoint, no desde cero.

Human-in-the-loop. Pausa antes de un paso sensible, una persona valida y el grafo continúa.

CASO REAL

Soporte de una tienda: por la mañana el cliente habla del pedido 1234, por la tarde vuelve y no repite nada.

EN CÓDIGO

```
app = g.compile(checkpointer=MemorySaver())
config = {"configurable": {"thread_id": "cliente-42"}}
```

MemorySaver: en RAM, ideal para practicar. Si reinicias, se borra.

SqliteSaver / Postgres: para producción. Cambiar de uno a otro es una línea.

IDEA CLAVE: son los puntos de guardado del videojuego.

thePower Tech School

LangChain vs LangGraph: cuándo usar cada uno

LangChain
La caja de herramientas: piezas listas.

- **Flujos lineales:** prompt va a modelo va a parser, de A a B.
- **Las piezas:** conectores, loaders, tools y parsers ya hechos.
- **RAG básico y prototipos:** algo funcionando en 20 líneas.

EJEMPLO: resume en una frase cada correo de soporte de hoy.

LangGraph
El jefe de cocina: organiza el servicio.

- **Agentes con estado:** el state compartido entre todos los pasos.
- **Bucles, ramas y memoria larga:** repetir, decidir y recordar.
- **Producción seria (2026):** reintentos y validación humana.

EJEMPLO: asistente que consulta, reintenta y pide validación.

REGLA 2026: no es uno u otro, es LangGraph por encima de LangChain.

thePower Tech School

El ciclo de un agente en LangGraph

1

El modelo piensa
Lee el state con todo el historial y decide qué hacer.
Ej.: "¿Cuál es el estado del pedido 1234?"

2

Decisión: tool o respuesta
La arista condicional lee la salida del modelo y elige camino.
Ej.: le faltan datos, va a tools.

3

Ejecuta el nodo de tools
Llama a la API o a la búsqueda y guarda el resultado en el state.
Ej.: consulta_pedido(1234).

4

Vuelve al modelo
Con el dato nuevo vuelve a pensar: repite o ya responde.
Ej.: "sale hoy, llega el jueves".

la flecha que vuelve = la arista condicional: repite las vueltas que hagan falta

La arista condicional crea el bucle: piensa, actúa, vuelve, y repite hasta poder dar la respuesta final. Esto es el patrón ReAct, ahora construido por dentro.

thePower Tech School

02

Bloque práctico

thePower
Tech School

Punto de partida: de la sesión 4 a la sesión 5

LO QUE YA TENÉIS

1. **Ollama + llama3.1**
   el mismo modelo local de la sesión 4

2. **Python + entorno virtual**
   un proyecto nuevo: langgraph-basics

3. **Claude Code**
   el mismo ciclo: pedir, revisar, ejecutar, entender

LO ÚNICO NUEVO

```
pip install langgraph
            langchain-core langchain-ollama
```

Sin descargas nuevas ni coste: langgraph es un paquete de Python.

LO QUE CAMBIA RESPECTO A LA SESIÓN 1

SESIÓN 1 Cadena LCEL: A -> B -> C

SESIÓN 2 **StateGraph: nodos, aristas y ciclos**

pasos 1-2

SESIÓN 1 RunnableWithMessageHistory (el aviso amarillo)

SESIÓN 2 **Checkpointer + thread_id**

paso 3

SESIÓN 1 Bucle de tools escrito a mano

SESIÓN 2 **ToolNode + arista condicional**

paso 4

thePower Tech School

Paso 1 · Grafo simple

QUÉ HACEMOS

- State mínimo: un TypedDict con un campo
- Dos nodos que transforman el state
- START -> nodos -> END y compilar
- Invocar y dibujar el grafo

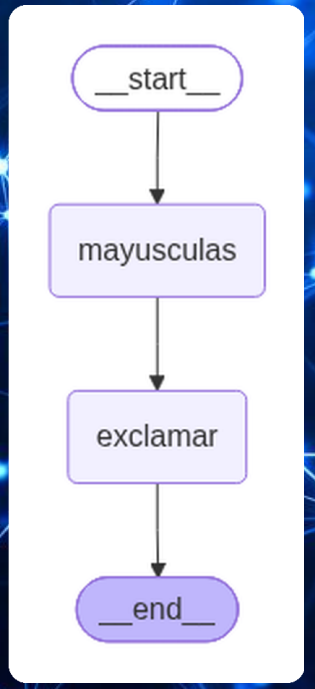
CONCEPTO CLAVE

Un grafo es State + nodos + aristas. Se compila, se invoca igual que una cadena y se puede dibujar.

```
>_ Windows PowerShell

(.venv) PS C:\Users\ignaciode\langgraph-basics> python 01_grafo_simple.py
ENTRADA : {'texto': 'hola langgraph'}
SALIDA  : {'texto': 'HOLA LANGGRAPH!!!'}
DIAGRAMA: guardado en grafo_simple.png

(.venv) PS C:\Users\ignaciode\langgraph-basics>
```



FÍJATE EN

cada nodo transforma el state y pasa el testigo: 'hola langgraph' sale como 'HOLA LANGGRAPH!!!'.

thePower Tech School

Paso 1 · Anatomía de un StateGraph

01_grafo_simple.py

```python
# 1) STATE: la "mochila" que viaja por el grafo. Aqui, un solo campo
class State(TypedDict):
    texto: str


# 2) NODOS: funciones que leen el state y devuelven SOLO las claves que cambian
def mayusculas(state: State) -> dict:
    return {"texto": state["texto"].upper()}


def exclamar(state: State) -> dict:
    return {"texto": state["texto"] + "!!!"}

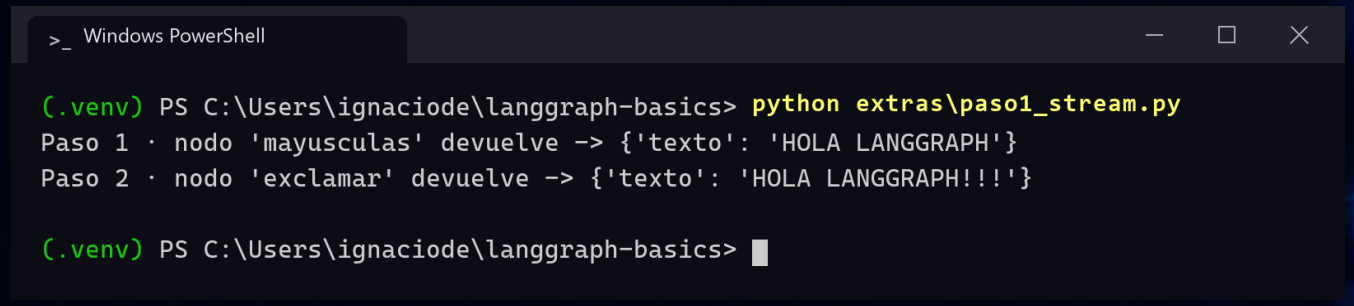

# 3) EDGES: que va despues de que. START y END marcan la entrada y la salida
builder = StateGraph(State)
builder.add_node("mayusculas", mayusculas)
builder.add_node("exclamar", exclamar)
builder.add_edge(START, "mayusculas")
builder.add_edge("mayusculas", "exclamar")
builder.add_edge("exclamar", END)

# 4) COMPILAR: el grafo compilado se ejecuta con invoke, igual que una cadena
grafo = builder.compile()
```

1. **State**
   un TypedDict: los datos que viajan por el grafo
2. **Nodos**
   funciones que leen el state y devuelven solo lo que cambian
3. **Edges**
   add_node y add_edge: qué va después de qué, de START a END
4. **Compilar**
   compile() y listo: se invoca igual que una cadena

thePower Tech School

Paso 1 · Variación: el grafo paso a paso



VARIACIÓN EN VIVO

QUÉ CAMBIAMOS

El mismo grafo ejecutado con stream en modo updates, para ver qué devuelve cada nodo en cada paso.

QUÉ DEMUESTRA

stream enseña el grafo por dentro: primero actúa mayusculas y luego exclamar, y cada uno devuelve solo la clave que cambia.

thePower Tech School

Paso 1 · Pregunta a la clase

PREGUNTA A LA CLASE

Si cambio el orden de las aristas (primero exclamar, luego mayusculas), ¿cambia el resultado? ¿Quién decide el orden?

Fijaos en que funciones y aristas se escriben por separado.

thePower Tech School

Paso 2 · Evolucionar a chatbot

QUÉ HACEMOS

- State con una lista de mensajes (add_messages)
- Un nodo que llama a ChatOllama
- START -> chatbot -> END
- Un turno de conversación

CONCEPTO CLAVE

El State pasa a ser una lista de mensajes: add_messages suma en vez de machacar y un nodo llama al modelo.

```
>_ Windows PowerShell

(.venv) PS C:\Users\ignaciode\langgraph-basics> python 02_chatbot.py
HumanMessage | Explícame en una frase qué es LangGraph.
AIMessage    | LangGraph es un proyecto de investigación que busca desarrollar un
    modelo de procesamiento de lenguaje natural basado en grafos, con el objetivo de
    mejorar la comprensión y el análisis del lenguaje humano.

Mensajes en el state al terminar: 2

(.venv) PS C:\Users\ignaciode\langgraph-basics>
```

FÍJATE EN

el grafo funciona... y el modelo se inventa qué es LangGraph. No lo conoce y responde con total seguridad.

thePower Tech School

Paso 2 · Anatomía del chatbot

02_chatbot.py

```python
# add_messages es un "reducer": SUMA los mensajes nuevos a la lista en vez de sustituirla  (1)
class State(TypedDict):
    messages: Annotated[list, add_messages]


modelo = ChatOllama(model="llama3.1", temperature=0)


# El nodo recibe TODO el historial y devuelve solo la respuesta nueva  (2)
def chatbot(state: State) -> dict:
    return {"messages": [modelo.invoke(state["messages"])]}


builder = StateGraph(State)  (3)
builder.add_node("chatbot", chatbot)
builder.add_edge(START, "chatbot")
builder.add_edge("chatbot", END)
grafo = builder.compile()
```

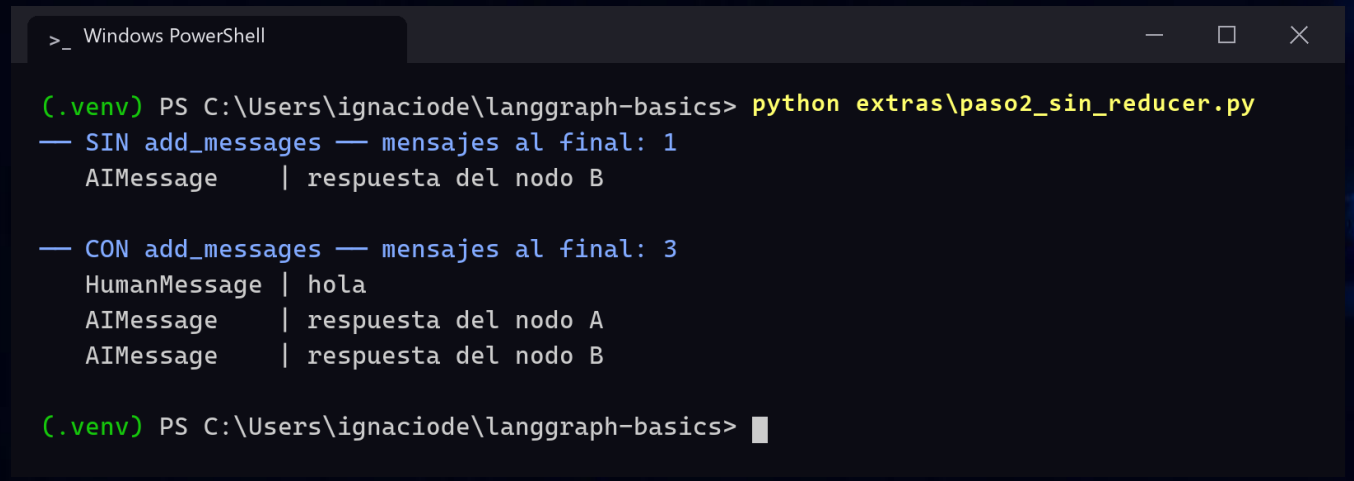
1. **add_messages**
   el reducer: suma los mensajes nuevos en vez de machacar la lista

2. **El nodo chatbot**
   recibe todo el historial y devuelve solo la respuesta nueva

3. **El cableado**
   mismo patrón del paso 1: un nodo entre START y END

thePower Tech School

Paso 2 · Variación: sin add_messages



VARIACIÓN EN VIVO

QUÉ CAMBIAMOS

Dos grafos idénticos, uno con add_messages en el State y otro sin él, y comparamos la lista final de mensajes.

QUÉ DEMUESTRA

Sin reducer, cada nodo sustituye la lista: queda 1 mensaje. Con add_messages se acumulan los 3. Una línea en el State decide si hay historial.

thePower Tech School

Paso 2 · Pregunta a la clase

PREGUNTA A LA CLASE

El grafo funciona, pero el modelo se inventó qué es LangGraph.
¿Cómo lo evitaríais?

El problema no está en el grafo.

thePower Tech School

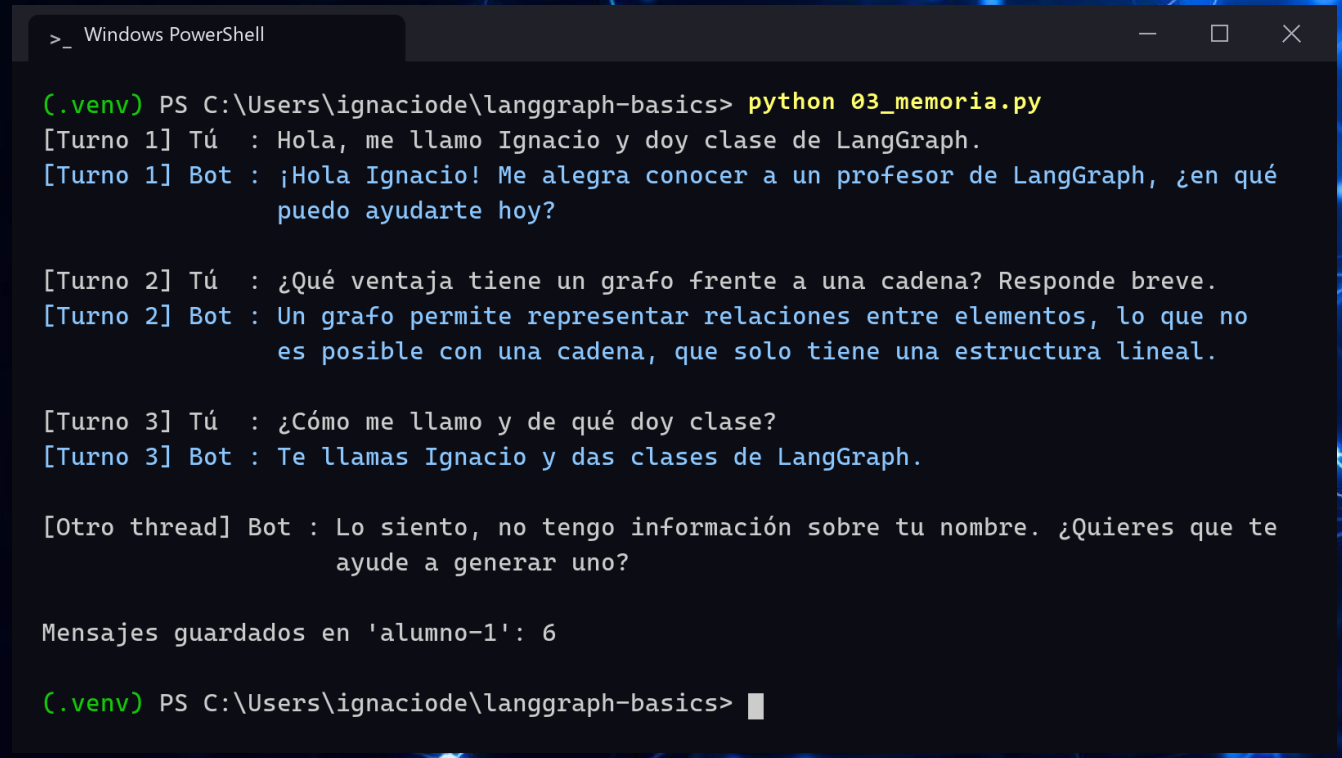
Paso 3 · Añadir memoria

QUÉ HACEMOS

- Compilar con un checkpointer MemorySaver
- Un thread_id por conversación
- 3 turnos: recuerda nombre y contexto
- Control: otro thread_id empieza de cero

CONCEPTO CLAVE

Compilar con un checkpointer guarda el state por thread_id: memoria entre turnos con una sola línea.



FÍJATE EN

mismo resultado que en la sesión 4, pero sin aviso amarillo: esta es la forma actual de dar memoria.

thePower Tech School

Paso 3 · Dónde vive la memoria

03_memoria.py

```python
# La UNICA diferencia con el paso 2: compilar con un checkpointer
grafo = builder.compile(checkpointer=MemorySaver())

if __name__ == "__main__":
    config = {"configurable": {"thread_id": "alumno-1"}}   # thread_id = una conversacion
    turnos = [
        "Hola, me llamo Ignacio y doy clase de LangGraph.",
        "¿Qué ventaja tiene un grafo frente a una cadena? Responde breve.",
        "¿Cómo me llamo y de qué doy clase?",
    ]
    for i, texto in enumerate(turnos, 1):
        salida = grafo.invoke({"messages": [HumanMessage(texto)]}, config=config)  # solo el mensaje NUEVO
        print(f"[Turno {i}] Tú  : {texto}")
        print(f"[Turno {i}] Bot : {salida['messages'][-1].content}\n")

    otro = grafo.invoke({"messages": [HumanMessage("¿Cómo me llamo?")]},
                        config={"configurable": {"thread_id": "alumno-2"}})
    print(f"[Otro thread] Bot : {otro['messages'][-1].content}")
    print(f"\nMensajes guardados en 'alumno-1': {len(grafo.get_state(config).values['messages'])}")
```

1. **El checkpointer**
   compile(checkpointer=MemorySaver()): la única línea nueva
2. **El thread_id**
   una conversación; mismo id sigue, id nuevo empieza en blanco
3. **Solo lo nuevo**
   cada turno envía solo el mensaje nuevo: el resto lo pone el checkpointer
4. **get_state**
   leer el state guardado de un thread en cualquier momento

thePower Tech School

Paso 3 · Variación: una foto en cada paso

```
>_ Windows PowerShell

(.venv) PS C:\Users\ignaciode\langgraph-basics> python extras\paso3_checkpoints.py
Checkpoints guardados en el thread: 6

  step -1 | input  | mensajes: 0 | siguiente nodo: __start__
  step  0 | loop   | mensajes: 1 | siguiente nodo: chatbot
  step  1 | loop   | mensajes: 2 | siguiente nodo: FIN
  step  2 | input  | mensajes: 2 | siguiente nodo: __start__
  step  3 | loop   | mensajes: 3 | siguiente nodo: chatbot
  step  4 | loop   | mensajes: 4 | siguiente nodo: FIN

(.venv) PS C:\Users\ignaciode\langgraph-basics>
```

VARIACIÓN EN VIVO

QUÉ CAMBIAMOS

Tras 2 turnos recorremos get_state_history: cada checkpoint con su step, origen, mensajes y siguiente nodo.

QUÉ DEMUESTRA

2 turnos = 6 checkpoints. Se guarda una foto del state antes y después de cada nodo: por eso se puede reanudar si algo se cae.

thePower Tech School

Paso 3 · Pregunta a la clase

PREGUNTA A LA CLASE

¿Qué diferencia hay entre add_messages y el checkpointer? ¿Hacen falta los dos?

Uno trabaja dentro de una ejecución; el otro, entre ejecuciones.

thePower Tech School

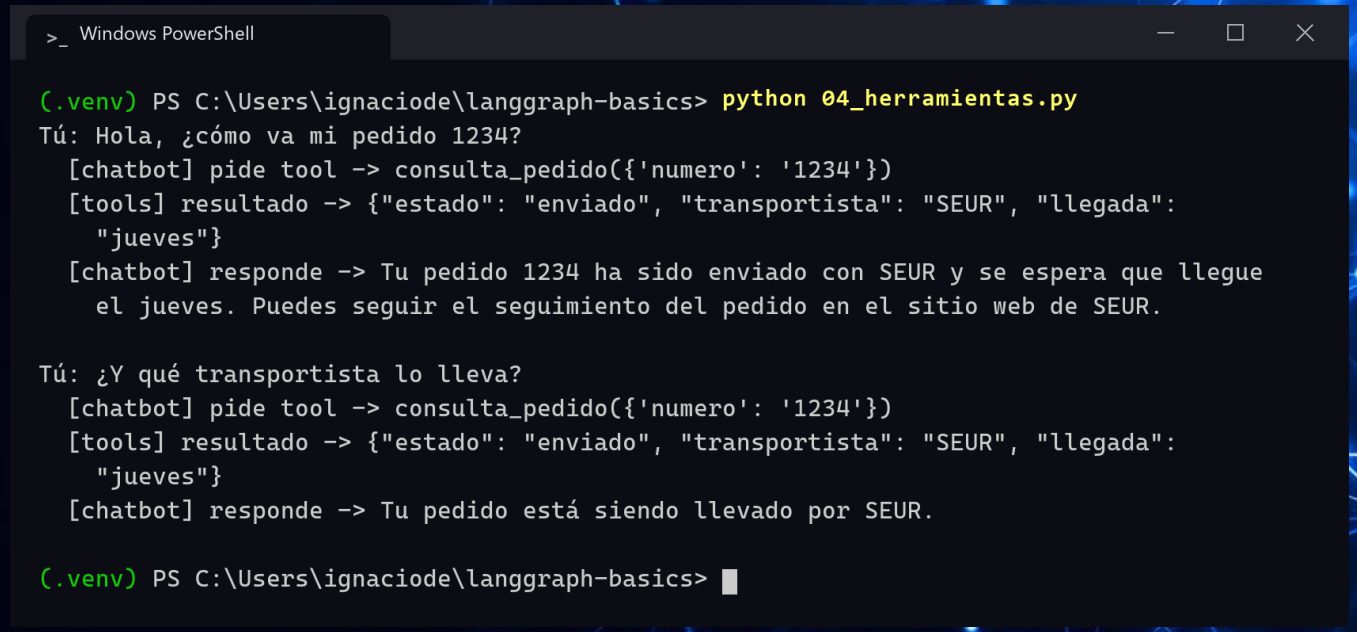
Paso 4 · Añadir herramientas

QUÉ HACEMOS

- Tool consulta_pedido con @tool
- ToolNode: el nodo que ejecuta tools
- Arista condicional: tool o respuesta
- Vuelta de tools al modelo: el ciclo

CONCEPTO CLAVE

ToolNode ejecuta las tools; la arista condicional decide entre tools o fin, y tools vuelve al modelo: el ciclo del agente.



FÍJATE EN

en el turno 2 no se dice el número de pedido: lo saca de la memoria. Tools y checkpointer trabajando juntos.

thePower Tech School

Paso 4 · El ciclo del agente en código

04_herramientas.py

```python
@tool
def consulta_pedido(numero: str) -> dict:
    """Devuelve el estado, el transportista y la fecha de llegada de un pedido a partir de su número."""
    return PEDIDOS.get(numero, {"error": f"No existe el pedido {numero}"})


tools = [consulta_pedido]


class State(TypedDict):
    messages: Annotated[list, add_messages]


modelo = ChatOllama(model="llama3.1", temperature=0).bind_tools(tools)
SYSTEM = SystemMessage("Eres el asistente de soporte de una tienda online. Responde en español, breve. "
                       "Usa la herramienta consulta_pedido cuando te pregunten por un pedido.")


def chatbot(state: State) -> dict:
    return {"messages": [modelo.invoke([SYSTEM] + state["messages"])]}


builder = StateGraph(State)
builder.add_node("chatbot", chatbot)
builder.add_node("tools", ToolNode(tools))                # nodo que EJECUTA las tools que pide el modelo
builder.add_edge(START, "chatbot")
builder.add_conditional_edges("chatbot", tools_condition)  # ¿pidio tool? -> "tools" ; si no -> END
builder.add_edge("tools", "chatbot")                       # tras la tool, vuelve al modelo: EL CICLO
grafo = builder.compile(checkpointer=MemorySaver())
```

1. **La tool**
   @tool + docstring: lo que el modelo lee para decidir
2. **bind_tools**
   el menú de herramientas pegado al modelo
3. **ToolNode**
   el nodo que ejecuta las tools que pide el modelo
4. **Condicional + vuelta**
   tools_condition elige camino; tools vuelve a chatbot: el bucle

thePower Tech School

Paso 4 · Variación: la arista decide

```
>_ Windows PowerShell

(.venv) PS C:\Users\ignaciode\langgraph-basics> python extras\paso4_sin_tool.py
Tú: Hola, ¿con qué me puedes ayudar?
  [chatbot] responde -> Lo siento, pero no tengo información sobre un pedido específico.
    ¿Podrías proporcionar el número de pedido para que pueda ayudarte mejor?

Tú: ¿Cómo va el pedido 5678?
  [chatbot] pide tool -> consulta_pedido({'numero': '5678'})
  [tools] resultado -> {"estado": "en preparación", "transportista": "-", "llegada": "la
    semana que viene"}
  [chatbot] responde -> El estado de tu pedido 5678 es "en preparación". No tenemos
    información sobre el transportista asignado, pero se espera que llegue la semana que
    viene.

(.venv) PS C:\Users\ignaciode\langgraph-basics>
```

VARIACIÓN EN VIVO

QUÉ CAMBIAMOS

Mismo grafo, dos preguntas: una sin pedido y otra sobre el pedido 5678, y comparamos el camino de cada una.

QUÉ DEMUESTRA

Sin pedido: chatbot y fin. Con pedido: chatbot, tools, chatbot. El mismo grafo toma caminos distintos según lo que decide el modelo.

thePower Tech School

Paso 4 · Pregunta a la clase

PREGUNTA A LA CLASE

En el turno 2 el agente volvió a consultar el pedido aunque ya tenía el dato. ¿Es un error?

¿Quién decide si se llama a la tool?

thePower Tech School

Paso 5 · Cierre y material

QUÉ HACEMOS

- Dibujar el grafo final con draw_mermaid_png
- Comentarios didácticos en el código
- README: State, nodos, aristas y checkpointer
- Repo listo para compartir

CONCEPTO CLAVE

El diagrama final resume el agente: la arista condicional punteada y el ciclo entre tools y chatbot.

```
Windows PowerShell

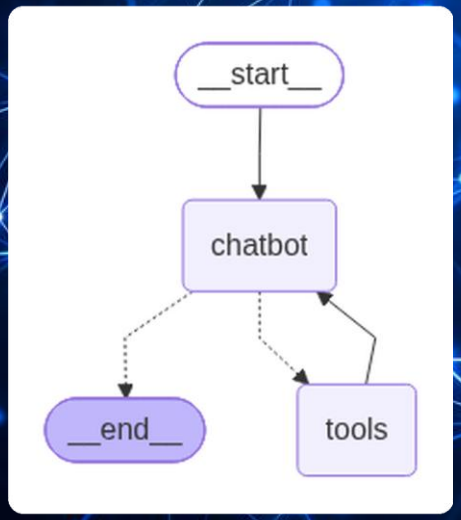
(.venv) PS C:\Users\ignaciode\langgraph-basics> python 05_grafo_final.py
Guardado grafo.png

(.venv) PS C:\Users\ignaciode\langgraph-basics> ls

    Directory: C:\Users\ignaciode\langgraph-basics

Mode   LastWriteTime       Length Name
----   -------------       ------ ----
d-----  23/09/2026 23:54:21        .venv
d-----  24/09/2026 00:00:33        extras
d-----  24/09/2026 00:02:20        soluciones
-a----  23/09/2026 23:59:55 1427   01_grafo_simple.py
-a----  23/09/2026 23:56:30 1153   02_chatbot.py
-a----  23/09/2026 23:56:35 1875   03_memoria.py
-a----  23/09/2026 23:56:43 2859   04_herramientas.py
-a----  24/09/2026 00:03:09 506    05_grafo_final.py
-a----  24/09/2026 00:03:12 8751   grafo.png
-a----  23/09/2026 23:59:56 8499   grafo_simple.png
-a----  24/09/2026 00:03:04 2480   README.md
-a----  23/09/2026 23:54:21 42     requirements.txt

(.venv) PS C:\Users\ignaciode\langgraph-basics>
```



FÍJATE EN

el diagrama final: las flechas punteadas son la arista condicional; la de tools a chatbot, el ciclo.

thePower Tech School

Recap: de la teoría al código

Concepto de la teoría	Dónde lo hemos usado	La línea clave
State	01 · todos	`class State(TypedDict): ...`
Nodos	01 · 02	`def chatbot(state): return {...}`
Edges	01 · 02	`add_edge(START, "chatbot")`
Reducer add_messages	02 · 03 · 04	`Annotated[list, add_messages]`
Checkpointer + thread_id	03 · 04	`compile(checkpointer=MemorySaver())`
Arista condicional + ciclo	04	`add_conditional_edges(...)` · `add_edge("tools", "chatbot")`

Lo que queda para más adelante: **human-in-the-loop** (pausar antes de un paso sensible). También se apoya en el checkpointer que hoy ya tenéis.

thePower Tech School

Ejercicio 1 · Un grafo que clasifica y responde

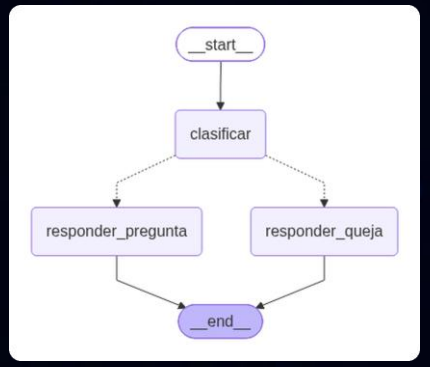
QUÉ CONSTRUIR

BÁSICO · 1,5 h

Un grafo mínimo de 2-3 nodos: un nodo clasifica el mensaje del usuario (**pregunta** o **queja**) y una arista condicional lo manda a un nodo de respuesta u otro.

PASOS SUGERIDOS

1. Definir el estado del grafo
2. Nodo clasificador
3. Arista condicional
4. Dos nodos de respuesta
5. Probar un mensaje de cada tipo



ENTREGABLE

Repositorio + un diagrama simple del grafo + 2 ejecuciones (una por rama).

```
>_ Windows PowerShell

(.venv) PS C:\Users\ignaciode\langgraph-basics> python soluciones\ejercicio1_clasificador.py
Mensaje  : ¿Hacéis envíos a Portugal?
Tipo     : pregunta  ->  rama responder_pregunta
Respuesta: Sí, nuestra tienda online ofrece envíos a Portugal, tanto a nivel nacional como
    a nivel internacional, con un plazo de entrega estimado de entre 3 a 7 días hábiles.

Mensaje  : Mi pedido llegó roto y nadie me contesta los correos.
Tipo     : queja  ->  rama responder_queja
Respuesta: Lo siento mucho, queremos asegurarte que te reembolsaremos el valor del
    producto roto y te ofrecemos un código de descuento del 10% para tu próxima compra
    para compensar el inconveniente. ¿Podrías por favor proporcionarnos tu número de
    pedido para que podamos proceder con el reembolso?

Diagrama guardado en grafo_clasificador.png

(.venv) PS C:\Users\ignaciode\langgraph-basics>
```

ASÍ DEBE VERSE

Ejecución real de la solución (soluciones/): cada mensaje va por su rama. Ojo: el bot se inventa políticas de envío y descuentos; sin datos reales improvisa.

thePower Tech School

Ejercicio 2 · Reto: el grafo por fases

RETO

QUÉ CONSTRUIR

1. **Grafo simple**
   State + nodos + edges
2. **Chatbot**
   add_messages + nodo con el modelo
3. **Memoria**
   checkpointer + thread_id
4. **Herramientas**
   tu propia tool + ToolNode + arista condicional

IDEAS PARA VUESTRA TOOL

`consulta_tiempo(ciudad)` *el tiempo de una ciudad (API pública)*

`buscar_en_csv(producto)` *precio y stock desde un CSV propio*

`reservar_mesa(dia, personas)` *guarda la reserva en un JSON*

La clave: una conversación donde el turno 2 dependa del turno 1 y de la tool. Así se ve que memoria y herramientas funcionan juntas.

ENTREGABLE

Repo con el grafo por fases + captura de una conversación multiturno usando una tool.

S6

PRÓXIMA SESIÓN · Observability

LangFuse y LangSmith: ver por dentro cada paso de estos grafos, con latencia, tokens y coste.

thePower Tech School